

Lecturer Details

By Mr. Baranga Peter

Msc. Information Systems

Bsc. Computer Science

Advanced. Certificate Network Security

0701-310089

0782-310089

barangaster@gmail.com

Outline

- Introduction to Programming
- Requirements
- Brief history of C programming language
- Steps of preparing a program
- The Program development life cycle
 - ✓ Creating the source code
 - ✓ Compiling the source code
 - ✓ Creating an executable file
 - ✓ Completing the development cycle
- Creating our first C program
- Recap of the lecture

Introduction to programming

What is a computer programming language:

- ✓ A computer language is a series of words used to give instructions to the computer, telling it what to do, how to do it, and when to do it.
- ✓ Just like there are various human languages used to communicate, there are also various computer languages.
- ✓ One of these computer languages is called C.
- ✓ In our lessons, we will learn C as a computer language not as a development environment.
- ✓ This means that we will primarily build applications that show how this language functions, not how to create graphical applications.

Requirements

What will be needed:

- ✓ Text editor. (Notepad, NotePad++....)
- ✓ Compiler. (Dev-Cpp, Gcc.....)
- ✓ Install a compiler on your Laptops!!

Brief History of C

- ✓ Created by Dennis Ritchie at the Bell Telephone Laboratories in 1972.
- ✓ Created to help in designing the UNIX operating system (which is used on many computers).
- ✓ Because C is such a powerful and flexible language, its use quickly spread beyond Bell Labs.
- ✓ The American National Standards Institute (ANSI) formed a committee in 1983 to establish a standard definition of C, which became known as ANSI Standard C.
- ✓ Every modern C compiler has the ability to adhere to this standard.
- ✓ **Now, what about the name?**
- ✓ The C language is so named because its predecessor was called B.
- ✓ The B language was developed by Ken Thompson of Bell Labs. You should be able to guess why it was called B.

Steps of preparing a program

- ✓ **You should take certain steps when you're solving a problem.**
- ✓ First, you must define the problem. If you don't know what the problem is, you can't find a solution!
- ✓ Once you know what the problem is, you can devise a plan to fix it.
- ✓ Once you have a plan, you can usually implement it.
- ✓ Once the plan is implemented, you must test the results to see whether the problem is solved.
- ✓ **This same logic can be applied to many other areas, including programming.**
- ✓ When creating a program in C (or for that matter, a computer program in any language), you should:
 - 1. Determine the objective(s) of the program.**
 - 2. Determine the methods you want to use in writing the program.**
 - 3. Create the program to solve the problem.**
 - 4. Run the program to see the results.**

- ✓ As an example, let's assume that someone asks you to write a program to determine the area of a circle.
- ✓ **Step 1** is complete, because you know your objective: **determine the area of a circle.**
- ✓ **Step 2** is to determine what you need to know in order to compute the area.
 - i. $A = \pi r^2$
 - ii. $\pi = 3.1416$
 - iii. Will the radius of the circle be provided or the user will input the radius he or she wants??
- ✓ Now you have the pieces you need, so you can continue to **steps 3 and 4.**
- ✓ The above steps make up what is called the **Program Development Cycle.**

The Program Development Life Cycle

✓ The Program Development Cycle has the following steps.

1. The first step, you use an editor to create a disk file containing your source code.
2. In the second step, you compile the source code to create an object file.
3. In the third step, you link the compiled code to create an executable file.
4. The fourth step is to run the program to see whether it works as originally planned.

Creating the Source Code

✓ What is a source code??

- *A Source code is a series of statements or commands that are used to instruct the computer to perform your desired tasks.*
- **E.g** `printf("My First C Program!!");`
- The above statement instructs the computer to **display** the message **My First C Program!!** on-screen. (For now, don't worry about how this statement works.)

✓ The source code is typed in an text editor. This is the area where the source code of any program is typed.

✓ Almost all modern day compilers come with a built-in editor that can be used to enter source code.

✓ When you save a source file, you must give it a name. The name should describe what the program does. **E.g circleArea or AreaOfCircle**

✓ When you save C program source files, give the file a **.C extension**. Although you could give your source file any name and extension, **.C** is recognized as the appropriate extension to use. **AreaOfCircle.c**

Compiling the Source Code

- ✓ As you may already be aware, A computer requires digital, or *binary*, instructions in what is called *machine language*.
- ✓ Before your C program can run on a computer, it must be translated from source code to machine language.
- ✓ This translation, the second step in program development, is performed by a
- ✓ program called a *compiler*.
- ✓ The compiler takes your source code file as input and produces a disk file containing the machine language instructions that correspond to your source code statements.
- ✓ The machine language instructions created by the compiler are called *object code*, and the disk file containing them is called an *object file*.
- ✓ **Note:** You compile a program to not only change the source code into machine language but to also *look for errors in your source code*.

Linking to create an executable file

- ✓ One more step is required before you can run your program.
- ✓ C contains a function library that contains *object code* (code that has already been compiled) for predefined functions.
- ✓ **A predefined function contains C code that has already been written and is supplied in a ready-to-use form with your compiler package.**
- ✓ The `printf()` function used in the previous example is a *library function*.
- ✓ *These library functions perform* frequently needed tasks, such as displaying information on-screen and reading data from disk files.
- ✓ If your program uses any of these functions (and hardly a program exists that doesn't use at least one), the object file produced when your source code was compiled must be combined with object code from the function library to create the final executable program. (*Executable means that the program can be run, or executed, on your computer.*)
- ✓ This process is called *linking*, and it's performed by a program called **a linker**.

Completing the Development Cycle

- ✓ Once your program is compiled and linked to create an executable file, you can run it
- ✓ If you run the program and receive results different from what you thought you would, you need to go back to the first step.
- ✓ You must identify what caused the problem and correct it in the source code.
- ✓ When you make a change to the source code, you need to recompile and relink the program to create a corrected version of the executable file.
- ✓ You keep following this cycle until you get the program to execute exactly as you intended.
- ✓ One final note on compiling and linking: Although compiling and linking are mentioned as two separate steps, many compilers, do both as one step.
- ✓ Regardless of the method by which compiling and linking are accomplished, understand that these two processes, even when done with one command, are two separate actions.

First C Program

```
#include <stdio.h>
```

```
main()
```

```
{
```

```
    printf("Programming is an interesting course unit\n");
```

```
    printf("but it requires a lot of practice in order to master it!!");
```

```
    return 0;
```

```
}
```

Errors

✓ Two main types of errors in programming:

1. Compilation Errors:

- ✓ A compilation error occurs when the compiler finds something in the source code that it can't compile.
- ✓ A misspelling, typographical error, or any of a dozen other things can cause the compiler to choke.
- ✓ Fortunately, modern compilers don't just choke; they tell you what they're choking on and where it is!
- ✓ This makes it easier to find and correct errors in your source code.

2. Linker Error Messages

- ✓ Linker errors are relatively rare and usually result from misspelling the name of a C library function. In this case, you get an Error: undefined symbols: error message, followed by the misspelled name (preceded by an underscore).
- ✓ Once you correct the spelling, the problem should go away.